

Types of AI

Demo 1 of 17 · Part one · What AI is · 20 minutes

You need: one browser and the demo's `index.html`; it works offline. Start from a fresh page or press Settings → Reset. Settings → Presentation mode scales the page for a projector; Open Presenter Notes shows this guide in the app.

Learning goals

Each stage's Takeaway, as the learner reads it:

1. **1a.** This opponent's skill came from a person: changing its rules changes its play, and playing more games changes nothing.
2. **1b.** Reinforcement learning stores experience as a separate score for each position; only an exhaustive search over every game line, not a good record, earns the word unbeatable.
3. **1c.** Shared weights give the network an answer for a board it never practised on — which a table cannot do — and training on other boards makes that answer better. The error readings say how close the answer is to the teacher's, not how well it plays.
4. **1d.** Architecture (how a model is built) and training method (how it learns) are separate choices: the same architecture can be trained more than one way, and the same method can train more than one architecture.
5. **Model types.** A model type names the job — what goes in and what comes out — not the architecture inside. One neural-network family, the transformer, serves text, vision, audio, video and multimodal types; generating images, audio and video also leans on a different family, diffusion models, which Hugging Face keeps in a library of its own.

Where this sits on the path

Demo 1 of 17 opens Part one. Nothing comes before it. It hands forward one idea to Demo 2 (How a Language Model Works): a model is a representation that can be trained, and *architecture*, *training method* and *model type* are three separate questions to ask of any AI.

One question for the session

Where does an AI's skill come from? Three models play the same board and each got its skill a different way. The top lens sorts models two ways: **Model architecture** (how it is built and trained — 1a to 1d) and **Model types** (what it does).

"Architecture is how it is built. Training method is how it learns. Type is the job. Keep asking which one someone means."

1a · Symbolic AI (GOFAI) — 4 minutes

Question

Can a program play well without ever learning?

Predict prompt

Ask the room to press a prediction: with only the **First 2** rules on, can you beat it? Take a show of hands before anyone plays.

Activity

1. **0:00** Press **First 2 rules**. Play one game; invite the room to find the fork. The Notice line and the panel name the rule that answered each move.
2. **1:30** Press **All 8 rules** and play again. Point at the rule that fires each turn.
3. **3:00** Open **Details**: the proof line reports the exhaustive search over every game playable against the ladder.

Expected observation

Two rules lose to a fork; eight rules have no losing line. The opponent explains every move by naming a rule, and nothing about it changes however many games are played.

Check answer: a person wrote every rule and its priority; playing did not change them. The wrong option ("it learned the rules by playing") is refuted by First 2 losing the same fork forever.

Teaching limit

GOF AI means Good Old-Fashioned Artificial Intelligence. This decision list can be drawn as a tree; here people wrote the tests, whereas decision trees can also be learned. Say "this small example", not "symbolic AI".

1b · Value table — 5 minutes

Question

Can experience alone teach a blank table to play?

Predict prompt

After one burst of 5,000 games, will Era 1 be unbeatable or still beatable? Press a prediction.

Activity

1. **0:00** In Era 0 press **Show move scores**: every square reads 0.00. Play one short game; it moves at random.
2. **1:00** Press **Train the AI**. Read the change report under the training controls: recorded scores before and after the burst.
3. **2:30** Train again until the Notice line says *verified unbeatable* (usually burst 3 to 5). Switch the **Opponent era** back to Era 0 and forward again: the report follows the selection.
4. **4:00** Open **Details** for the proof line.

Expected observation

Era 1 is still beatable; the scores moved but a losing line remains. A later era has no losing line in either role. Human play never changes the table; only the burst does.

Check answer ("Does playing the AI above train it?"): no — here only the training burst updates the table; your games are scored, not learned from. Say the caveat out loud: that is this demo's choice, not a law. The same table

could have learned from every game you played, and real assistants can use what people type to train later versions, usually behind a setting. Details cites two vendors' own pages (fetched 2026-09-16).

Teaching limit

A prediction of “unbeatable” is a claim about the table, settled by the exhaustive search, not by whether the room beat it: losing to a beatable era looks exactly like losing to an unbeatable one, and the reveal line says which one the search found. The table has one separate score per position, so it carries nothing to a different situation. Tic-tac-toe is small enough to tabulate; chess and language are not, which is why the next stage shares parameters.

1c · Neural Network — 6 minutes

Question

Can one set of shared weights score boards it never practised on?

Predict prompt

After training, will held-out error be about the same as practised error, or clearly higher? Press a prediction; the reveal quotes the measured numbers.

Activity

1. **0:00** Press **Create learning examples**. A fresh table learner plays 20,000 seeded games; its estimates freeze as targets. This is the one slow step.
2. **1:30** In the network view, pick a connection layer and a neuron; read one weight and its initialized value.
3. **2:30** Press **Train network**. Compare the weight with its initial value, then the practised and held-out errors, then the separate playing score. **Show move scores** shows predictions for legal moves.
4. **5:00** Optional: press **Ultimate tic-tac-toe** above the board to show a representation that cannot see the whole game; **Standard board** brings the network back.

Expected observation

The network is a feedforward multilayer perceptron (29 inputs, 28 and 18 ReLU units, one output) fitted by supervised learning to targets another model produced. Backpropagation supplies gradients; updates are normalized gradient steps. Shared weights give an answer for a held-out position; the error readings say how close, and neither is playing strength.

Check answers: error measures agreement with the frozen table's scores, which are themselves estimates; play is measured separately. 1b stores one number per position; 1c reuses 1,381 shared parameters for every board.

Teaching limit

Lower error means closer agreement with the teacher, not guaranteed playing strength; no automatic unbeatable claim is made. A position the fresh teacher never visited stays at zero — a limit of the examples, not proof the position is neutral. During play the network calculates its own scores without consulting the frozen table. Activations are responses to a board, not stored weights; a positive weight is not a good move.

1d · Other architectures & training methods — 3 minutes

Question

What else can a model be, and how else can it learn?

Predict prompt

None on this stage. 1d is a map, not an experiment; nothing on it runs a result a prediction could be checked against. If you want a show of hands anyway, ask which of 1a, 1b, 1c learned from inputs paired with target answers, then press the **Supervised learning** card.

Activity

1. **0:00** The stage has three parts behind three buttons. Under **Training methods**, press each card. The card faces do not say which model used them; the press does: supervised → 1c, reinforcement → 1b, no training → 1a; the other three light nothing. Point at the one-line “used for” notes: pretraining is self-supervised, fine-tuning and distillation are supervised, RLHF is reinforcement — stages, not extra methods.
2. **1:30** Switch to **Other architectures** and press a card: the line beneath says what that model stores and what it computes — linear models, decision trees, ensembles, nearest neighbours, support-vector machines, probabilistic models. Search and planning are named separately as problem-solving approaches.
3. **2:15** Switch to **Learning with no weight change**: in-context learning, retrieval, search at decision time. Tie it to 1b’s check: a model can appear to learn in a session without changing. Press **Search at decision time**: 1a’s fork rules look one move ahead every turn; the demo’s exhaustive search checks players and never picks a move.

Expected observation

Architecture and training method are separate columns. 1c is a network trained by supervised learning; a network could equally be trained by reinforcement, and a table could be fitted to supervised targets.

Check answer: no — the network is an architecture; supervised learning is how this one was trained.

Teaching limit

The cards are textbook categories, not a complete catalogue, and real systems combine them. Self-play is not self-supervision: 1b learns from rewards. Gradient training is not evolution.

Model types — 3 minutes

Question

If “type” means the job, what goes in and what comes out?

Predict prompt

None on this stage: it is informational, and nothing on it runs a result a prediction could be checked against.

Activity

1. **0:00** Switch the top lens to **Model types**. Press three cards the room will recognise: text generation, text-to-speech, speech-to-text (automatic speech recognition). Each shows what goes in and comes out, then the catalogue’s own definition.

2. **1:30** Press **Image-to-3D**: the job people call image-to-mesh. Follow **Browse models** on one card if you are online; the demo itself never fetches.
3. **2:00** Scroll to **Beyond one catalogue**: eight major types the Hub list does not show — world models (Genie 3), video with sound (Veo 3.1), robot control (Gemini Robotics), protein structure (AlphaFold), weather (GenCast), background removal (NVIDIA Broadcast), character animation, free-viewpoint video — each linked to the model's own page.
4. **2:30** Scroll to the whole catalogue: every type Hugging Face lists, in its groups, with the model count on the fetch date.

Expected observation

A type is named by its input and output. Text generation and image-to-text are different types that can share one architecture family; tabular types are the visible exception, where the catalogue names tree and boosting libraries. The link back to 1c: a small network scoring boards is the same idea as the networks behind most of these types.

Check answer: not necessarily — type is the job; one transformer may generate text and another caption images.

Teaching limit

The taxonomy and the counts are Hugging Face's, fetched on the date shown on the stage; counts move daily. Other catalogues cut the jobs differently. The stage does not run any of these models.

Misconceptions

"A program that plays well must have learned."

1a plays unbeatable tic-tac-toe with no learning at all. Skill can be written down by a person; learning is one way to get it, not the definition of AI.

"A long run of draws proves it is unbeatable."

Sampled games can miss a line. The app calls an era unbeatable only after searching every reachable game line in both roles. Open Details for the counts.

"Playing against the table trains it."

Not here: human play is scored, not learned from, and every era is a frozen copy. But it is a design choice, not a rule — real systems can and sometimes do train on what users type.

"I lost, so it must be unbeatable."

Beatable means a losing line exists, not that you found it. The search settles the verdict; your record only says how you played.

"Low error means it plays well."

Error is agreement with a teacher whose estimates are imperfect. The playing score is a separate, fixed-seed measurement, and this tab never claims unbeatable.

“Neural network is a kind of learning.”

It is an architecture. 1c’s network was fitted by supervised learning; the same weights could be trained by reinforcement. Press the method cards in 1d.

“Same architecture, same type.”

Type is the job. Hugging Face files transformers under text generation, image-to-text, speech recognition and more. Ask what goes in and what comes out.

Discussion questions

- Which model stores separate values, and which shares weights? What does each choice make easy and hard?
- Why does lower held-out error not prove unbeatable play? What measurement would?
- Name an AI product you use. Which type is it, what might its architecture be, and how do you think it was trained? Which of the three can you actually tell from the outside?
- The Ultimate extension: what information does the inherited representation leave out, and why can more training not put it back?

Presenter controls

- **Guide** (header) reopens the learner card. **Settings** has Open Presenter Notes, Presentation mode and Reset, then a playing preference, a rehearsal seed, the self-test and About.
- **Reset** is in place: stage 1a, First 2 rules, move scores hidden, Era 0, records and predictions cleared, type cards closed. Presentation mode and the playing preference stay.
- **Rehearsal seed**: type a seed and press Reset; training then unfolds identically, so a rehearsal matches the lesson.
- **Run the full self-test** re-runs the exhaustive search on the selected stage and prints it in Settings and the console.
- Deep links: #stage-1 to #stage-5; the older #rules, #learning, #neural still work.